

GENAI FOR TEST MANAGEMENT • TRACK 2

Module 1

GenAI Fundamentals & AI-Assisted PM Workflows

Slides 1-10 • Hands-on lab included

What Is Generative AI?

Definition: AI that produces brand-new content — text, images, video, audio — from a simple description

1 Classification Models

Sort and categorize existing information — e.g., filtering spam vs. inbox.

2 Generative Models

Create new content from learned patterns — e.g., drafting a report from a few key points.

This is the branch of AI that Copilot Chat belongs to.

How Do LLMs Work?

Just enough to explain why hallucinations happen

1

Trained on huge amounts of text

Books, articles, code — broken into small units called tokens.

2

Predicts the next token by probability

One token at a time — not by looking up facts.

3

Uses a transformer neural network

Good at spotting patterns across long text (mention only, no math needed today).

4

This is why hallucinations happen

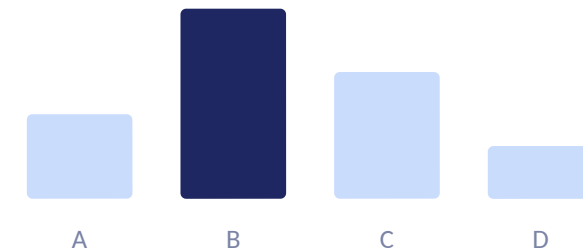
The model makes "its best guess at every step," not a fact lookup.

PREDICTING THE NEXT TOKEN



Model predicts the highlighted token from everything before it

CANDIDATE PROBABILITIES



Why GenAI for PMs: Use Cases vs. Hype

OK Good fit

- Summarizing sprint/backlog text
- Drafting first-pass reports
- Spotting patterns across notes

NO Not a fit

- A source of truth
- A replacement for judgment
- Reliable on numbers it wasn't given

As Slide 2 explained — this shows up in Copilot too: "output that sounds plausible but is factually incorrect... or entirely fabricated." — GitHub, "Hallucination," Responsible use of Copilot Chat

Takeaway: treat GenAI as a fast first draft, not a final answer.

What Is GitHub Copilot Chat?

1

Definition

The AI-powered chat interface for GitHub Copilot — GitHub's own term for it.

2

Where you'll use it today

VS Code (Chat view) and Copilot Chat on github.com.

3

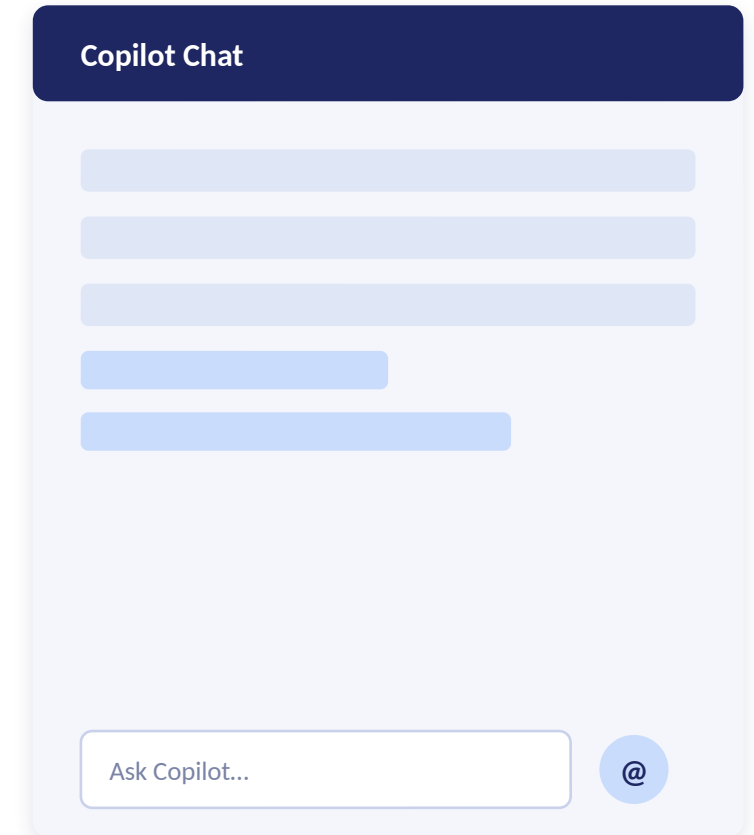
Also available in

GitHub Mobile, CLI, JetBrains, Xcode — mention only, not used today.

4

Key point for PMs

Chat, attached files, and custom instructions work the same for text as for code.



Prompt Engineering Foundations

Five official GitHub techniques — practiced today, in this order

- 1 Start general, then get specific
- 2 Give examples
- 3 Break big asks into steps
- 4 Be precise — name the file/data source explicitly
- 5 Iterate — don't settle for the first draft

BONUS TIP

New thread, new task

Copilot Chat uses chat history as context. Start a fresh thread per task so old context doesn't leak in.

Context Is Everything: Feeding Copilot the Right Inputs

Context = the files/data you attach, not just the wording of the prompt

PATTERN

1

Attach the sprint/backlog export

2

Reference it explicitly in the prompt

3

Ask for the specific artifact you need

NO

Anti-pattern

Asking for a summary with nothing attached —
Copilot will guess, not report.

Status Reports & Risk Reviews with Copilot

1 Status Report Pattern

Sprint data + an example of a "good" report + an explicit ask ("draft this week's update in this format").

2 Risk Review Pattern

Backlog + meeting notes → Copilot surfaces risk candidates (blocked items, ageing tickets, scope changes) → you validate.

Copilot surfaces candidates — the Test Manager decides what's actually a risk.

Sprint & Retrospective Summaries with Copilot

Same foundational techniques from Slide 5 apply here — same pattern, different input. Keep the ask decomposed: summary first, action items second.

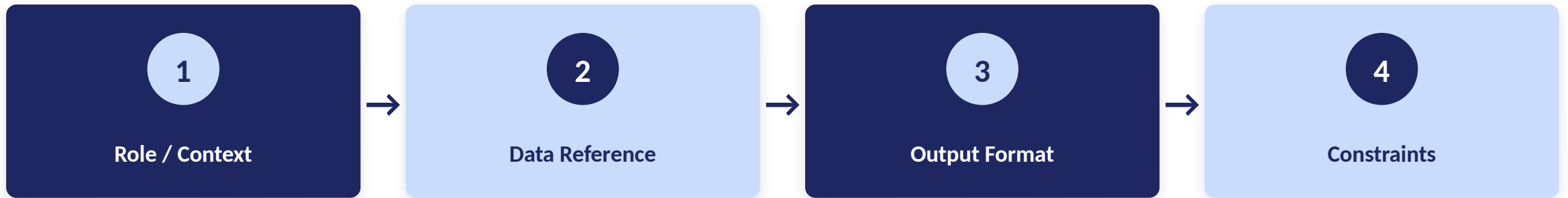
Went Well

Didn't Go Well

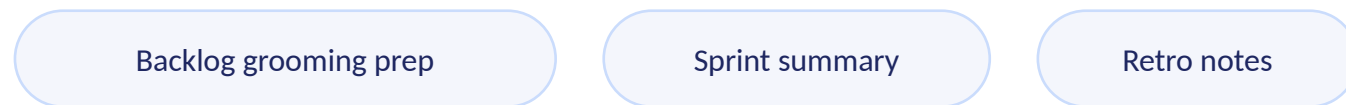
Action Items

Reusable Prompt Templates for Agile Ceremonies

Build one by taking a weak attempt, noting the correction, and baking it into the wording.



USE CASES



Module 1 Lab



Practice the 5 prompt techniques (Slide 5) on the provided sample sprint data



Generate a status report (Slide 7) and a retrospective summary (Slide 8) with Copilot Chat



Refine your weakest attempt into a reusable template (Slide 9)



Lab data pending

The lab folder is currently empty. This exercise needs a sprint export, a backlog export, a meeting-notes file, and a retro-notes file before it can run as written.